Project Initialization and Planning Phase

Date	07 July 2026
Team ID	xxxxxx
Project Title	Credit Card Approval Prediction
Maximum Marks	5 Marks

Project Proposal (Proposed Solution) report

This proposal outlines a machine-learning approach to credit-card approval decisioning, replacing slow, inconsistent manual and rule-based review with a data-driven classification model. The project builds a full pipeline — from raw applicant and credit-bureau data through a vintage-corrected target label, feature engineering, cross-validated model tuning, and deployment — culminating in a Flask web application that returns a real-time Approve/Reject decision with a disclosed confidence level.

Project Overview	
Objective	Build and deploy a supervised machine-learning model that predicts whether a credit-card applicant is likely to become a good (approved) or risky (rejected) account, using application-form data such as income, employment, housing, and family structure — and serve that prediction through a web interface for real-time decision support
Scope	Covers the full pipeline: collecting and understanding two linked datasets (applicant records and monthly credit-bureau records), exploratory visualization, construction of a reliable target label via vintage (cohort) analysis, data preprocessing and encoding, comparison of baseline classifiers, a cross-validated hyperparameter-tuned XGBoost model, and deployment as a Flask web application.
Problem Statement	
Description	Manual and static rule-based credit-approval review is slow and inconsistent across reviewers. Compounding this, naive labeling of "risky" applicants (based on ever having a serious delinquency, regardless of how long the account had been observed) mislabels short-history applicants as safe simply because they haven't been tracked long enough — baking label noise into the very data used to train and evaluate approval models.
Impact	Correcting this labeling bias with a vintage-filtered target (only applicants with $\geq$24 months of observed history are labeled) produced a cleaner, more trustworthy signal: cross-validated test F1 for the reject class improved from 0.345 to 0.410 (~+19% relative), with precision and recall improving together. This translates into faster, more consistent approve/reject decisions, reduced risk from

	mislabeled approvals, and underwriters freed to focus on genuinely borderline cases.
Proposed Solution	
Approach	Compare baseline classifiers (Logistic Regression, Decision Tree, Random Forest, XGBoost) with default hyperparameters as an informational benchmark, then run a cross-validated (5-fold × 40-candidate) RandomizedSearchCV hyperparameter search for XGBoost, optimized for F1 on the minority "reject" class. Select the operating decision threshold from out-of-fold probability estimates rather than the default 0.5, and validate once on an untouched hold-out test set. Serve the final model through a Flask web form for single-applicant, real-time predictions.
Key Features	1)Vintage-analysis-based label construction (≥24 months-on-book threshold) that removes label noise from prematurely-observed accounts. 2)Engineered features — age, years employed, employment-status flag, dependents ratio, family size — plus one-hot/ordinal encoding of categorical fields and income outlier capping. 3)Baseline comparison across four algorithms, followed by cross-validated hyperparameter tuning of XGBoost with an F1-optimized, out-of-fold decision threshold (0.29). 4)Real-time single-applicant prediction via a Flask form, returning an Approve/Reject decision with its probability and the threshold used. 5)Transparent disclosure of the model's actual precision/recall on the result page, so predictions are not presented as more reliable than they are.

	- Real-time decision-making for quicker loan approvals. - Continuous learning to adapt to evolving financial landscapes.

Resource Requirements

Resource Type	**Description**	**Specification/Allocation**
Hardware		
Computing Resources	CPU/GPU specifications, number of cores	CPU only — no GPU required. XGBoost/scikit-learn training on this tabular dataset (~13K labeled rows, 46 features) runs comfortably on a standard 4+ core CPU.
Memory	RAM specifications	8 GB
Storage	Disk space for data, models, and logs	~1 GB SSD (raw datasets ~67 MB, cleaned dataset ~1.7 MB, model artifacts <1 MB, plus headroom for logs/environment)
Software		
Frameworks	Python frameworks	Flask
Libraries	Additional libraries	pandas, numpy, scikit-learn, xgboost, joblib, matplotlib, seaborn
Development Environment	IDE	Jupyter Notebook (EDA), PyCharm / VS Code (application & pipeline scripts)
Data		
Data	Source, size, format	Kaggle "Credit Card Approval Prediction" dataset-application_record.csv (438,557 applicant rows) and credit_record.csv (1,048,575 monthly credit-bureau rows), csv format. After linking, 36,457 applicants have a credit-bureau history; 13,153 of those meet the ≥24-month vintage threshold and form the

		modeling dataset.